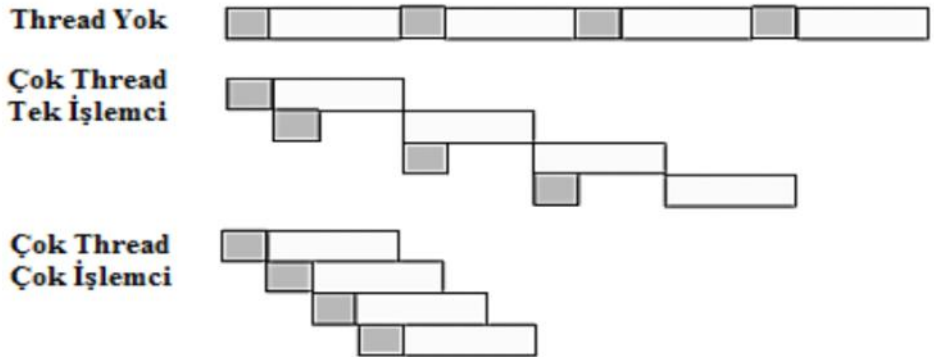
thread ve Paralel Programlama

27 Nisan 2026 Pazartesi 10:10

2 thread kullanılması pratikte tam olarak süreyi yarıya düşürmese de oldukça verimlilik sağlar.

Her kod paralelleştirilemez. Paralelleştirmeden önce şu sorular sorulmalıdır:
ilk soru kod paralelleştirilebilir mi?
Hangi tür paralelleştirme gereklidir veya gerekli midir?

donanımsal paralellik:



Bir uygulamayı çok çekirdeğe bölmek iyi bir yaklaşım değildir. Paralellik durumu varsa genelde tek çekirdek çok thread yaklaşımı uygulanır. Biz derste daha çok yazılımsal paralelliği kullanıcaz.

Fazla thread iyi demek değildir. Gereksiz kullanılmış threadler uygulamayı yavaşlatır.

Eş Zamanlılık Türleri:

4 farklı türü vardır:

- **Makine komutu düzeyinde :** 2 veya daha fazla makine komutunun paralel çalıştırılması
- **Program kod satırı düzeyinde:** 2 veya daha fazla program kod satırının paralel çalıştırılması
- **Birim seviyesinde :** 2 veya daha fazla fonksiyonun paralel çalıştırılması
- **Program seviyesinde :** 2 veya daha fazla programın paralel çalıştırılması

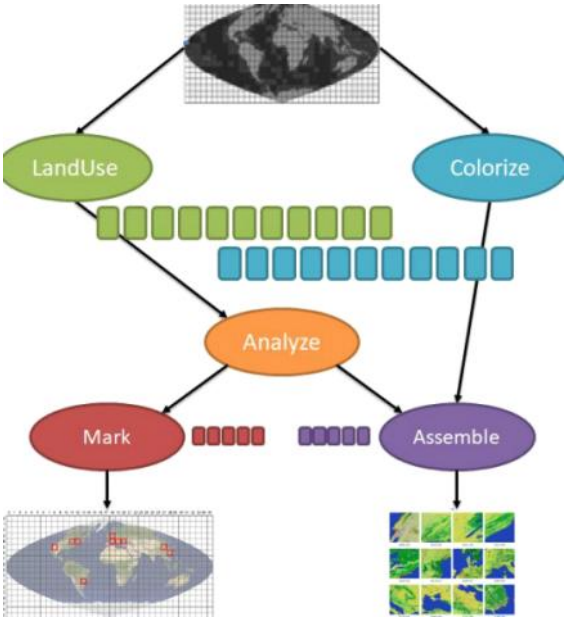
Biz program kod satırı düzeyine odaklanıcaz.

Hiç ekstra thread yoksa yani paralellik yoksa 1 adet thread vardır o da Main thread'dir.
2 tane eklenirse 3 adet thread olur.

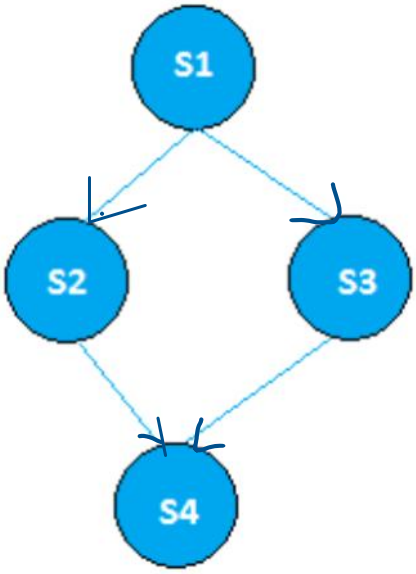
Örneğin c#'ta for döngüsü için Parallel.for(…){} gibi otomatik paralelleştiren özellikleri vardır.
Manuel olarak parallelleştirmek her zaman çok daha etkilidir. Manuel yapmak hata ve beklenmeyen davranışları önler.
Java direkt kendi kütüphanesi olarak destekler.
Bir çok dilde de kütüphaneler kullanılarak yapılabilir.

Derste kod olarak pratikte sadece javadan sorumluyuz.

Paralelleştirme noktasında öncelik grafları oluşturulur ve ona göre bir paralelleştirme planı yapılır.
örn:

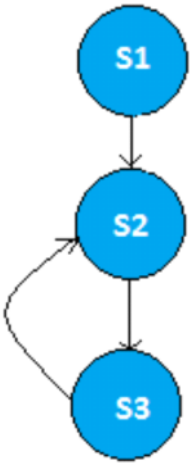


- Bağımlı ve bağımsız değişkenlerin tespit edilip hangi işlerin eş zamanlı çalıştırılabileceği belirlenir.
- Bir öncelik grafı hangi işlemlerin eş zamanlı hangi işlemlerin seri olarak çalıştırılması gerekeceğini gösterir.
- Hangi işlemlerin hangi işlemleri bekleyeceği görülebilir.



- S2 ve S3 çalıştırılabilmesi için S1 ifadesi işlemini bitirmelidir.
- S2 ve S3 eş zamanlı çalıştırılabilir.
- S4 çalışabilmesi için hem S2 hem de S3 işlemini bitirmelidir.

hatalı öncelik grafı örneği (deadlock):



Eş zamanlılık Şartları:

R(Si) = {a1, a2, … , an}: Si için "oku" kümesi.

W(Si) ={b1, b2, … , bn}:Si için "yaz" kümesi.

S1 ve S2 eşzamanlı çalışması için aşağıdaki 3 şart gerçekleşmelidir.

1. $R(S1) \cap W(S2) = \{\}$
2. $W(S1) \cap R(S2) = \{\}$
3. $W(S1) \cap W(S2) = \{\}$

Her ikisi aynı anda yazamaz biri okurken diğeri yazamaz.
İkisi de değer okuyabilir sorun yok.

S1 a:=x + y;
S2 b:= z + 1;
S3 c:= a - b;

R(S1)={x, y}
R(S2)={z}
R(S3)={a, b}
W(S1)={a}
W(S2)={b}
W(S3)={c}

S1 ve S2 deyimleri eş zamanlı olarak çalışabilir mi?
Koşul 1.R(S1)∩ W(S2)= {x,y}∩{b}={}
Koşul 2.W(S1) ∩ R(S2)={a}∩ {z}= {}
Koşul 3.W(S1) ∩ W(S2)={a}∩ {b}= {}

S1 ve S3 deyimleri eş zamanlı olarak çalışabilir mi?
Koşul 1.R(S1) ∩ W(S3)= {x,y} ∩{c}={}
Koşul 2.W(S1) ∩ R(S3)={a} ∩ {a,b}= {a}
Koşul 3.W(S1) ∩ W(S3)={a} ∩ {c}= {}

S2 ve S3 deyimleri eş zamanlı olarak çalışabilir mi?
Koşul 1.R(S2) ∩ W(S3)= {z}∩{c}={}
Koşul 2.W(S2) ∩ R(S3)={b}∩ {a,b}= {b}
Koşul 3.W(S2) ∩ W(S3)={b}∩ {c}= {}

Fonksiyonel paradigmalı dillerde paralel programlama:

- Multi-LISP
  - 1985 Yılında tanıtılan bu dil program parçalarının eş zamanlı çalışmasına izin veriyordu.
  - pcall yapısı ile bu gerçekleştiriliyordu.

(fonk x y z) → Eş Zamanlılık olmayan normal bir fonksiyon çağrımı

(pcall fonk x y z) → Eş Zamanlılık içeren fonksiyon çağrımı

  - pcall ile çağrılması x, y ve z parametrelerinin eş zamanlı çalışmasını sağlayacaktır. x y ve z parametreleri yine bir fonksiyon olabilir bu durumda bu fonksiyonlar eş zamanlı çalıştırılır.

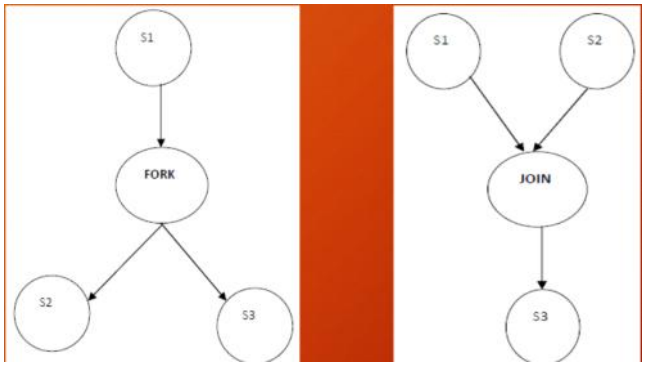
Paralelleştirme Yapıları

27 Nisan 2026 Pazartesi 10:41

Burdaki örnekler öncelik grafı örnekleridir

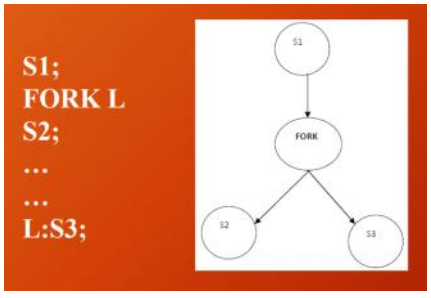
For ve Join Yapısı:

iki farklı örnek:

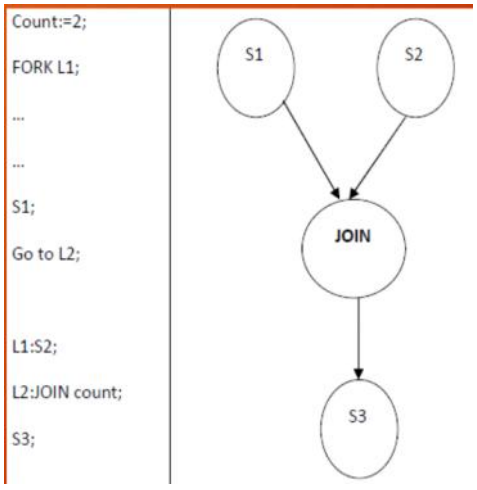


join 2den fazla işlemi de birleştirilebilir. Bekleme işlemi yapar join 4 dersen 4 adet işlemin bitmesini bekler. Programı tekrar seri hale sokar. Programlar seri başlar seri biter.

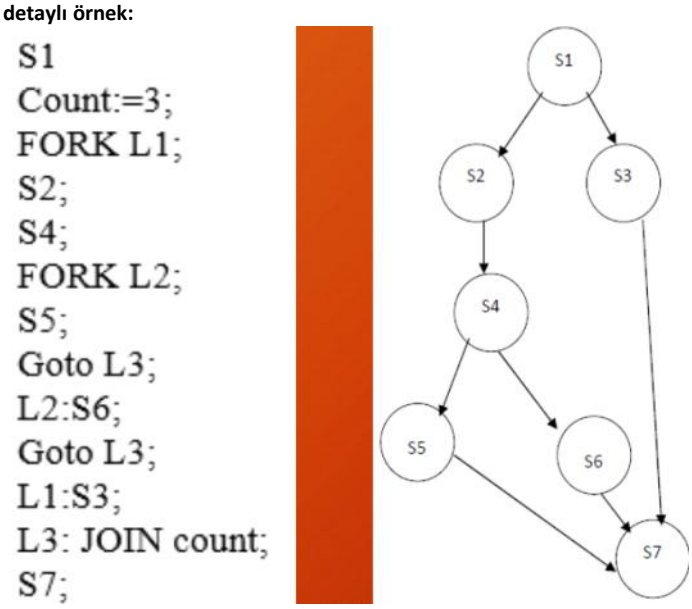
Fork:



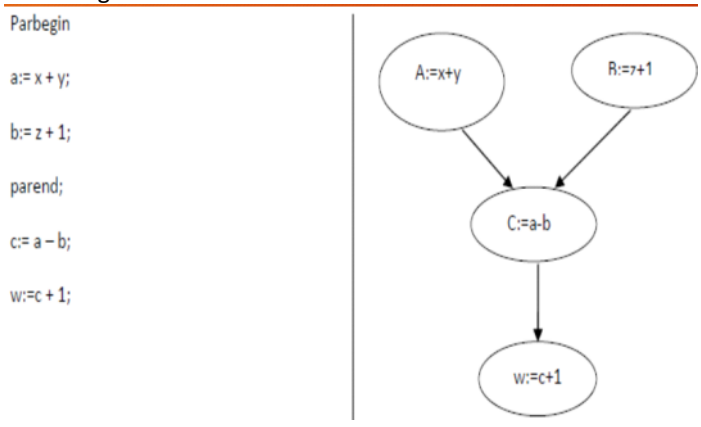
Join:



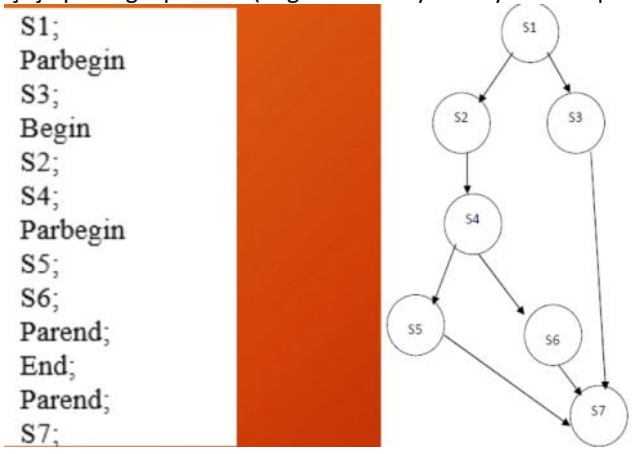
detaylı örnek:



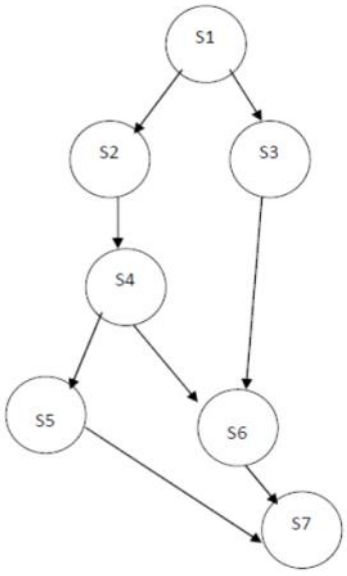
Parbegin Parend:



iç içe parbegin parend: (begin end ekleyerek ayrı bir scope açıyoruz)



Fork Join ile yapılabilen Parbegin Parend ile yapılamayan örnek:




```
S1;
Count1:=2;
FORK L1;
S2;
S4;
Count2:=2;
FORK L2;
S5;
Goto L3;
L1:S3;
L2:JOIN count1;
S6;
L3:JOIN count2;
S7;
```

bu örnekte: s3 ile s2, s4 ve s5(kısmen) paralel fakat s6 kesinlikle paralel değil.

thread ve yield

27 Nisan 2026 Pazartesi 11:24

thread yield metodu:

• yield metodu geçici olarak diğer threadlere zaman verir.
• Thread.yield(); şeklinde kullanılır.
• O satıra gelen her thread bu komutu uygulayacaktır.

thread sleep metodu:

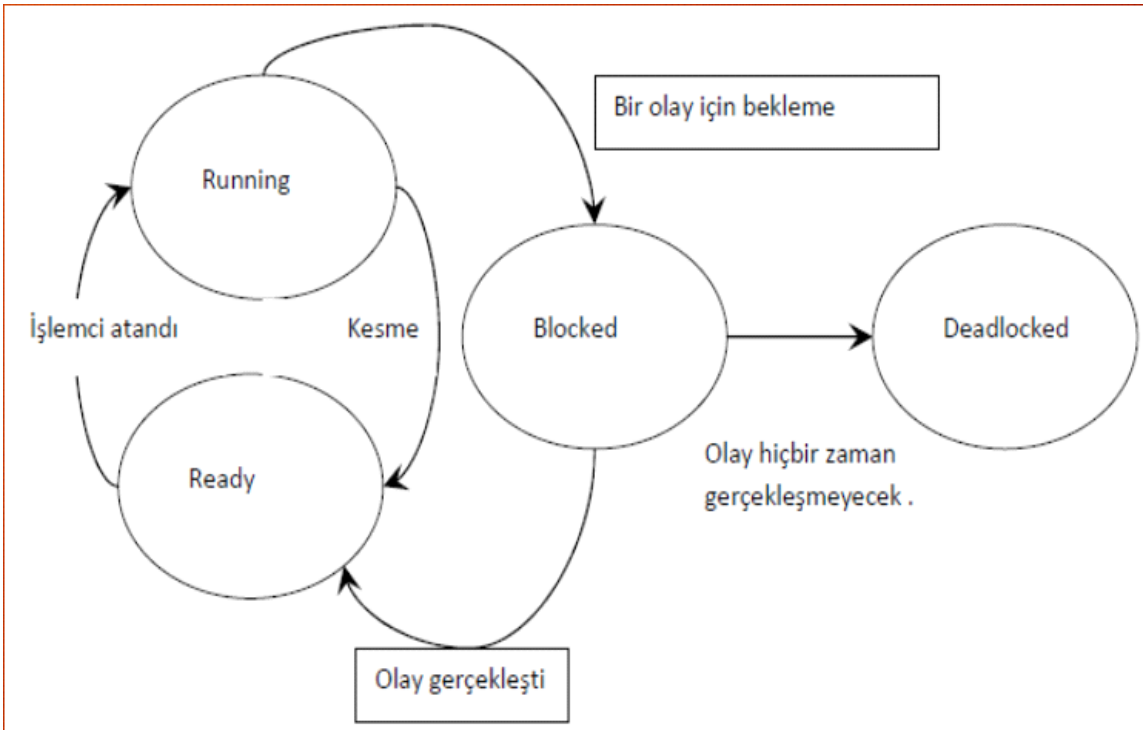
• Belirtilen süre boyunca thread uykuya geçer.
• Bu tepki süresinin uzun olduğu bazı durumlarda zorunlu olarak kullanılabilmektedir.
• Thread.sleep(1); 1 milisaniye uykuya geçirir.
• Java dilinde sleep metodu yakalanması zorunlu bir hata fırlatma durumu olduğu için try catch bloklarında kullanımı zorunludur.

• Bu metotların kullanımı için eş zamanlılık şart değildir.
• Programa atanan varsayılan thread bu komutlara denk geldiğinde işleyişi uygular.

İşlem Durumları ve Grafları

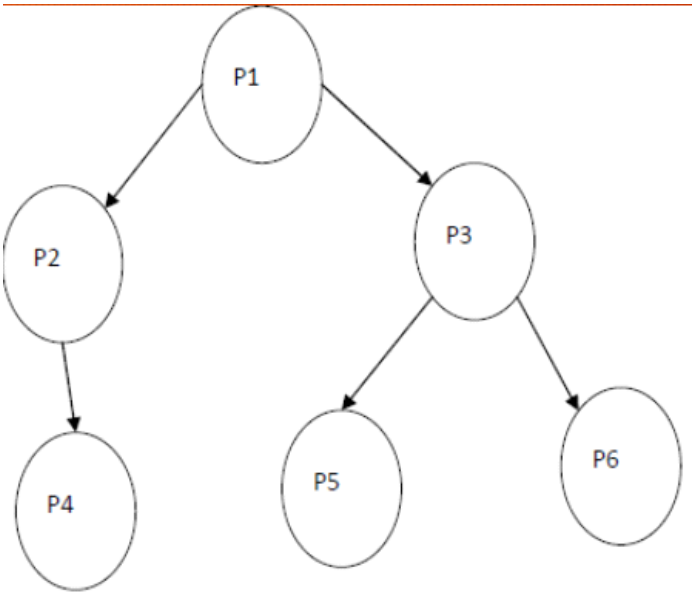
27 Nisan 2026 Pazartesi 11:27

İşlem Durumları:



- Running: Komutlar işletiliyor.
- Blocked: Sistem bazı durumlar için bekletiliyor.
- Ready: İşlem bir işlemciye atanmak için hazır durumda bekletiliyor.
- Deadlock: İşlem hiç bir zaman gerçekleşmeyecek olayları bekliyor

İşlem Grafları:



- Bir işlem grafı düğümleri işlemlere karşılık gelen yönlendirilmiş köklü bir graftır.
- Pi düğümünden Pj düğümüne gelen ok işareti Pi ’nin Pj ’yi oluşturduğunu ifade eder

Kritik Bölge

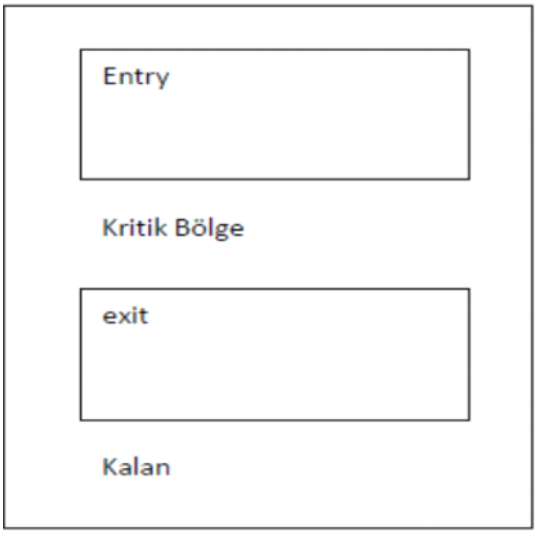
27 Nisan 2026 Pazartesi 11:32

- Birlikte çalışan n adet işlemden {P1, P2, …,Pn} oluşan bir sistem olduğu düşünülürse.
- Her bir işlem ortak değişkenleri okuyan bir tabloyu güncelleyen, bir dosyayı yazan vb. işlemleri içerebilir.
- Bu bölümlere kritik bölge ismi verilir.
- Bu bölgelere aynı anda sadece bir thread girmelidir.

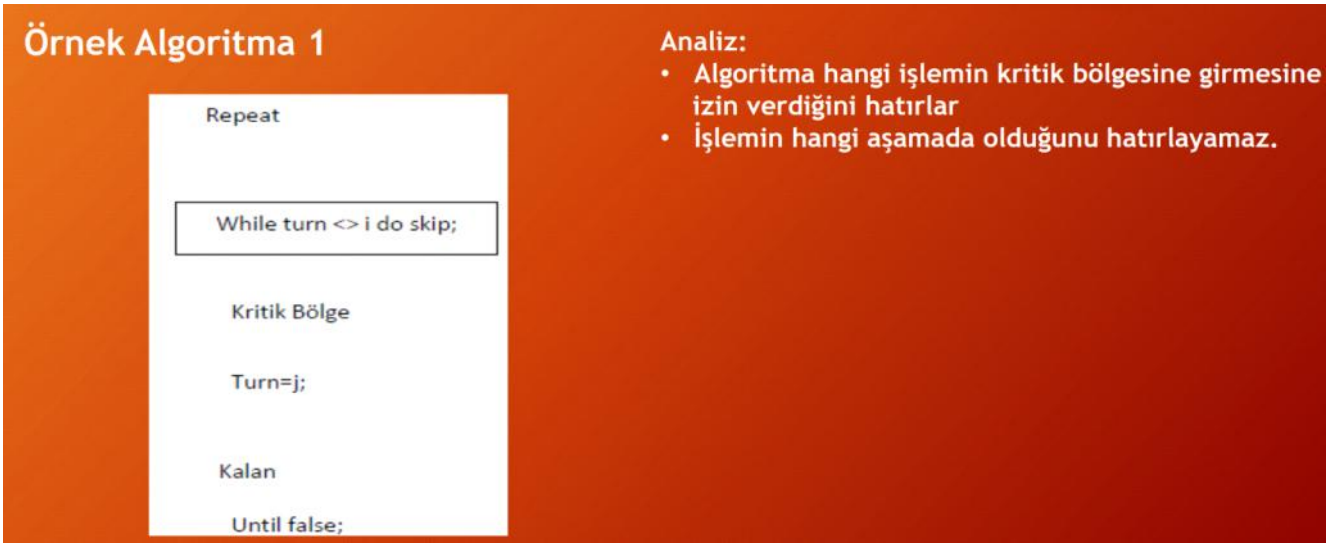
Çok uzun ve paralelleştirmenin iyi olacağı kodlarda paralelliğe engel olan kısımlar kritik bölgeye alınır. Kritik bölge içerisinde tek bir thread bulunabilir.

Javada bu otomatik halledilir. Bunu manuel olarak nasıl yaparız incelicez.

Kritik bölge yapısı:



Yaklaşımlar:



Örnek Algoritma 2

```
Repeat
    While flag[j] do skip;
        Flag[i]=true;
    Kritik Bölge
    Flag[i]:=False;
    Kalan
    Until false;
```

Analiz:

- Algoritma hangi işlemin kritik bölgesine girmesine izin verdiğini hatırlar
- İşlemin hangi aşamada olduğunu hatırlar.
- Fakat bir dizi kontrol edildiği için aynı anda birden fazla thread'in kritik bölgeye girme ihtimali vardır.

Örnek Algoritma 3

```
Repeat
    flag[i]:=true;
    turn:=j;
    While (flag[j] and turn=j)do skip;
    Kritik Bölge
    Flag[i]:=False;
    Kalan
    Until false;
```

Analiz:

- Algoritma hangi işlemin kritik bölgesine girmesine izin verdiğini hatırlar
- İşlemin hangi aşamada olduğunu hatırlar.
- Birden fazla thread'in kritik bölgeye girmesine ihtimal yoktur.

Semaforlar:

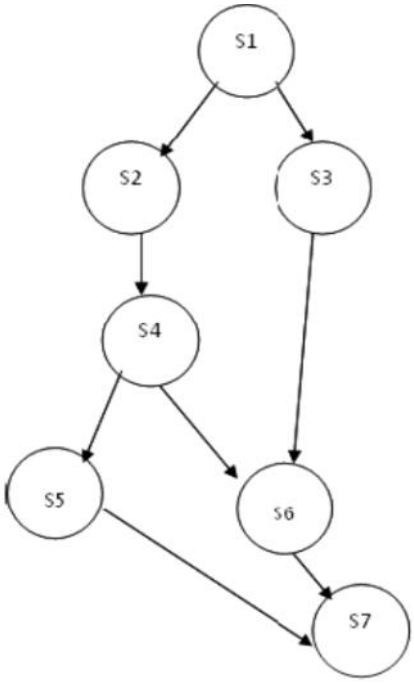
- Kritik bölgeyi çözmek için kullanılan yöntemlerden biridir.
- Karşılıklı hariç bırakma (mutual exclusuion) problemi için yapılan çözümleri daha komplex problemler için genelleştirmek kolay değildir.
- Bu zorluğun üstesinden gelebilmek için semaforlar olarak adlandırılan bir senkronizasyon aracı kullanılabilir.

```
Repeat
    P(Multex)
    Kritik Bölge
    V(Multex)
    Kalan
    Until false;
```

S1; V(synch);	P1 işlemine konulur.
P(synch); S2;	P2 işlemine konulur.

p ile içeri girer v ile çıkar. P'den içeri girme izni alır V'ye ben çıktım mesajı verir.

Parbegin parend ile semaforun kullanılması:




```
Var a, b, c, d, e, f, g: semaphore;
Begin
Parbegin
Begin S1; V(a); V(b) end;
Begin P(a); S2; S4; V(c); V(d); end;
Begin P(b); S3; V(e); end;
Begin P( c); S5; V(f); end;
Begin P(d); P(e); S6; V(g); end;
Begin P(f); P(g); S7; end
Parend;
End;
```

Referanslar/Kaynaklar:

Programlama Dillerinin Prensipleri Hafta 12 – Eş Zamanlı Programlama Doç. Dr. M. Fatih ADAK ders notları pdf